

---

# Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

## Experiment 2

### Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |

**Name:** T. A. Prasanna

**Roll No.:** 24110124

**Registration No.:** 24011101113

**GitHub Link:** <https://github.com/Prasannnnnnnnnaaaaa/Deep-Learning-Lab.git>

## 1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

## 2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.
7. Explain why a hidden layer is required to solve non-linearly separable problems such as XOR.

## 3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

---

## 4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size:  $28 \times 28$

## 5. Image Preprocessing

Fashion-MNIST images are stored as  $28 \times 28$  matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Preprocessing performed:

1. Loaded Fashion-MNIST via `keras.datasets.fashion_mnist.load_data()`.
2. Displayed sample images.
3. Flattened each  $28 \times 28$  image into a 784-length vector.
4. Normalized pixel values from  $[0, 255]$  to  $[0, 1]$ .
5. Converted integer labels into one-hot vectors using `keras.utils.to_categorical`.

## 6. Experimental Procedure

### Task 1: Dataset Exploration

- Loaded Fashion-MNIST.
- Printed dataset dimensions: `x_train: (60000, 28, 28), x_test: (10000, 28, 28)`.
- Displayed ten sample images.
- Plotted class distribution.

**Expected Output:** Dataset summary and sample images.

### Task 2: Data Preprocessing

- Flattened images from  $(28, 28)$  to  $(784, )$ .
- Normalized pixel values to  $[0, 1]$ .
- Printed tensor shapes before and after preprocessing.

### Task 3: Model Construction

Constructed an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

`model.summary()` output:

### Task 4: Model Training

Compiled using

- Optimizer: Adam
  - Loss: Categorical Cross Entropy
  - Metric: Accuracy
- Trained for 20 epochs with batch size 32.

### Task 5: Model Evaluation

Computed on the baseline model:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

## 7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, automated hyperparameter optimization was performed using **RandomizedSearchCV** (recommended over an exhaustive Grid Search given the size of the search space), together with the SciKeras wrapper (equivalently, a manual Randomized Search with 5-fold cross-validation implemented directly in TensorFlow/Keras).

Search space optimized:

| Hyperparameter          | Candidate Values    |
|-------------------------|---------------------|
| Hidden Layers           | 1, 2, 3             |
| Hidden Neurons          | 32, 64, 128, 256    |
| Learning Rate           | 0.1, 0.01, 0.001    |
| Batch Size              | 16, 32, 64, 128     |
| Epochs                  | 10, 20, 30          |
| Optimizer               | SGD, Adam, RMSProp  |
| Activation Function     | ReLU, Tanh, Sigmoid |
| Dropout Rate (Optional) | 0.0, 0.2, 0.5       |

### Tasks Performed

1. Built a baseline MLP model.
2. Defined the hyperparameter search space.
3. Performed Randomized Search using 5-fold cross-validation over 15 randomly sampled candidate configurations, on a stratified subsample of the training set.
4. Recorded the best hyperparameter combination.
5. Retrained the model on the full training set using the optimized hyperparameters.
6. Evaluated the optimized model on the held-out testing dataset.
7. Compared the optimized model with the baseline model (see Section 9, Performance Comparison).

## 1 Source Code

The complete implementation (dataset loading, preprocessing, baseline model, training, evaluation, hyperparameter optimization, and plotting) is given below.

```

1 import random
2 import time
3
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8 import tensorflow as tf
9 from tensorflow import keras
10 from tensorflow.keras import layers
11
12 from sklearn.model_selection import KFold
13 from sklearn.metrics import (
14     accuracy_score, precision_score, recall_score, f1_score,
15     confusion_matrix, classification_report
16 )
17
18 RANDOM_SEED = 42
19 np.random.seed(RANDOM_SEED)
20 tf.random.set_seed(RANDOM_SEED)
21 random.seed(RANDOM_SEED)
22
23 CLASS_NAMES = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
24               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
25
26 #Task 1
27 (x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()
28
29 print(f"x_train shape: {x_train.shape}, y_train shape: {y_train.shape}")
30 print(f"x_test shape : {x_test.shape}, y_test shape : {y_test.shape}")
31 print(f"Number of classes: {len(np.unique(y_train))}")
32 print(f"Pixel value range: [{x_train.min()}, {x_train.max()}]")
33
34 fig, axes = plt.subplots(2, 5, figsize=(12, 5))
35 for i, ax in enumerate(axes.flat):
36     ax.imshow(x_train[i], cmap='gray')
37     ax.set_title(CLASS_NAMES[y_train[i]], fontsize=9)
38     ax.axis('off')
39 fig.suptitle("Sample Fashion-MNIST Images")
40 plt.tight_layout()
41 plt.savefig('plot_1_sample_images.png', dpi=600)
42 plt.close()
43 print("Inference: The dataset consists of 28x28 grayscale clothing images "
44       "spanning 10 visually distinct categories. Some classes (e.g., shirt "
45       "vs. pullover vs. coat) look visually similar, which is a likely "
46       "source of misclassification.")
47 classes, counts = np.unique(y_train, return_counts=True)
48 plt.figure(figsize=(9, 5))
49 sns.barplot(x=[CLASS_NAMES[c] for c in classes], y=counts, palette='viridis')
50 plt.xticks(rotation=45, ha='right')
51 plt.ylabel("Number of training samples")
52 plt.title("Class Distribution (Training Set)")
53 plt.tight_layout()
54 plt.savefig('plot_2_class_distribution.eps', dpi=600)
55 plt.close()
56 print("Inference: All 10 classes contain exactly 6000 training samples "
57       "each, so the dataset is perfectly balanced and no class-imbalance "
58       "handling is required.\n")
59
60 #Task 2
61 print(f"Before preprocessing -> x_train: {x_train.shape}, dtype: {x_train.dtype}")
62
63 x_train_norm = x_train.astype('float32') / 255.0
64 x_test_norm = x_test.astype('float32') / 255.0
65
66 x_train_flat = x_train_norm.reshape(x_train_norm.shape[0], -1)
67 x_test_flat = x_test_norm.reshape(x_test_norm.shape[0], -1)
68
69 num_classes = 10
70 y_train_oh = keras.utils.to_categorical(y_train, num_classes)

```

```

71 y_test_oh = keras.utils.to_categorical(y_test, num_classes)
72
73 print(f"After preprocessing -> x_train_flat: {x_train_flat.shape}, "
74       f"y_train_oh: {y_train_oh.shape}")
75 print(f"After preprocessing -> x_test_flat : {x_test_flat.shape}, "
76       f"y_test_oh : {y_test_oh.shape}\n")
77
78 #Task 3
79 def build_baseline_model():
80     model = keras.Sequential([
81         layers.Input(shape=(784,)),
82         layers.Dense(128, activation='relu'),
83         layers.Dense(64, activation='relu'),
84         layers.Dense(10, activation='softmax')
85     ])
86     model.compile(optimizer='adam', loss='categorical_crossentropy',
87                  metrics=['accuracy'])
88     return model
89
90
91 baseline_model = build_baseline_model()
92 baseline_model.summary()
93 print()
94
95 #Task 4
96 start_time = time.time()
97 history = baseline_model.fit(
98     x_train_flat, y_train_oh,
99     validation_split=0.1,
100     epochs=20,
101     batch_size=32,
102     verbose=2
103 )
104 baseline_train_time = time.time() - start_time
105 print(f"Baseline training time: {baseline_train_time:.2f} seconds\n")
106 plt.figure(figsize=(8, 5))
107 plt.plot(history.history['accuracy'], label='Training Accuracy')
108 plt.xlabel('Epoch'); plt.ylabel('Accuracy')
109 plt.title('Training Accuracy vs Epoch'); plt.legend()
110 plt.tight_layout()
111 plt.savefig('plot_3_training_accuracy.eps', dpi=600)
112 plt.close()
113 print("Inference: Training accuracy rises steadily and plateaus in later "
114       "epochs, showing the model is learning discriminative features from "
115       "the flattened pixel inputs.")
116 plt.figure(figsize=(8, 5))
117 plt.plot(history.history['val_accuracy'], label='Validation Accuracy', color='orange')
118 plt.xlabel('Epoch'); plt.ylabel('Accuracy')
119 plt.title('Validation Accuracy vs Epoch'); plt.legend()
120 plt.tight_layout()
121 plt.savefig('plot_4_validation_accuracy.eps', dpi=600)
122 plt.close()
123 print("Inference: Validation accuracy closely tracks training accuracy "
124       "with a small gap, indicating the baseline model generalizes "
125       "reasonably well without severe overfitting.\n")
126 plt.figure(figsize=(8, 5))
127 plt.plot(history.history['loss'], label='Training Loss')
128 plt.xlabel('Epoch'); plt.ylabel('Loss')
129 plt.title('Training Loss vs Epoch'); plt.legend()
130 plt.tight_layout()
131 plt.savefig('plot_5_training_loss.eps', dpi=600)
132 plt.close()
133 print("Inference: Training loss decreases monotonically across epochs, "
134       "confirming that Adam is effectively minimizing the categorical "
135       "cross-entropy objective.")
136
137 plt.figure(figsize=(8, 5))
138 plt.plot(history.history['val_loss'], label='Validation Loss', color='orange')
139 plt.xlabel('Epoch'); plt.ylabel('Loss')
140 plt.title('Validation Loss vs Epoch'); plt.legend()
141 plt.tight_layout()

```

```

142 plt.savefig('plot_6_validation_loss.eps', dpi=600)
143 plt.close()
144 print("Inference: A plateauing (rather than rising) validation loss "
145       "suggests the model has converged well without significant "
146       "overfitting.\n")
147
148 #Task 5
149 y_pred_prob = baseline_model.predict(x_test_flat, verbose=0)
150 y_pred = np.argmax(y_pred_prob, axis=1)
151 y_true = np.argmax(y_test_oh, axis=1)
152
153 baseline_acc = accuracy_score(y_true, y_pred)
154 baseline_prec = precision_score(y_true, y_pred, average='macro')
155 baseline_rec = recall_score(y_true, y_pred, average='macro')
156 baseline_f1 = f1_score(y_true, y_pred, average='macro')
157
158 print(f"Baseline Test Accuracy : {baseline_acc:.4f}")
159 print(f"Baseline Precision(mac): {baseline_prec:.4f}")
160 print(f"Baseline Recall(macros) : {baseline_rec:.4f}")
161 print(f"Baseline F1-score(mac) : {baseline_f1:.4f}")
162 print("\nClassification Report (Baseline):")
163 print(classification_report(y_true, y_pred, target_names=CLASS_NAMES))
164
165 cm = confusion_matrix(y_true, y_pred)
166 plt.figure(figsize=(9, 7))
167 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
168            xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES)
169 plt.xlabel('Predicted'); plt.ylabel('Actual')
170 plt.title('Confusion Matrix - Baseline Model')
171 plt.tight_layout()
172 plt.savefig('plot_7_confusion_matrix_baseline.eps', dpi=600)
173 plt.close()
174 print("Inference: Most confusion occurs between visually similar apparel "
175       "classes such as Shirt, T-shirt/top, Pullover and Coat, while "
176       "distinct classes like Trouser, Bag and Sandal are classified with "
177       "high accuracy.\n")
178
179 #Task 6
180 def build_tunable_model(hidden_layers, hidden_neurons, learning_rate,
181                        activation, optimizer_name, dropout_rate):
182     model = keras.Sequential()
183     model.add(layers.Input(shape=(784,)))
184     for _ in range(hidden_layers):
185         model.add(layers.Dense(hidden_neurons, activation=activation))
186         if dropout_rate > 0.0:
187             model.add(layers.Dropout(dropout_rate))
188     model.add(layers.Dense(10, activation='softmax'))
189
190     if optimizer_name == 'adam':
191         opt = keras.optimizers.Adam(learning_rate=learning_rate)
192     elif optimizer_name == 'sgd':
193         opt = keras.optimizers.SGD(learning_rate=learning_rate)
194     else:
195         opt = keras.optimizers.RMSprop(learning_rate=learning_rate)
196
197     model.compile(optimizer=opt, loss='categorical_crossentropy',
198                 metrics=['accuracy'])
199     return model
200
201 SEARCH_SPACE = {
202     'hidden_layers': [1, 2, 3],
203     'hidden_neurons': [32, 64, 128, 256],
204     'learning_rate': [0.1, 0.01, 0.001],
205     'batch_size': [16, 32, 64, 128],
206     'epochs': [10, 20, 30],
207     'optimizer_name': ['sgd', 'adam', 'rmsprop'],
208     'activation': ['relu', 'tanh', 'sigmoid'],
209     'dropout_rate': [0.0, 0.2, 0.5],
210 }
211 }
212

```

```

213 N_ITER = 15
214 N_FOLDS = 5
215 SEARCH_SAMPLE_SIZE = 6000
216
217 rng = np.random.RandomState(RANDOM_SEED)
218 sub_idx = rng.choice(x_train_flat.shape[0], SEARCH_SAMPLE_SIZE, replace=False)
219 x_search = x_train_flat[sub_idx]
220 y_search_int = y_train[sub_idx]
221 y_search_oh = y_train_oh[sub_idx]
222
223 random.seed(RANDOM_SEED)
224 candidates = []
225 for _ in range(N_ITER):
226     cand = {k: random.choice(v) for k, v in SEARCH_SPACE.items()}
227     candidates.append(cand)
228
229 kfold = KFold(n_splits=N_FOLDS, shuffle=True, random_state=RANDOM_SEED)
230
231 search_results = []
232
233 print(f"Running manual RandomizedSearch: {N_ITER} candidates x {N_FOLDS}-fold CV "
234       f"= {N_ITER * N_FOLDS} model fits on a {SEARCH_SAMPLE_SIZE}-image subsample...\n")
235
236 search_start = time.time()
237 for c_idx, cand in enumerate(candidates, start=1):
238     fold accuracies = []
239     for fold_idx, (tr_idx, val_idx) in enumerate(kfold.split(x_search), start=1):
240         model = build_tunable_model(
241             hidden_layers=cand['hidden_layers'],
242             hidden_neurons=cand['hidden_neurons'],
243             learning_rate=cand['learning_rate'],
244             activation=cand['activation'],
245             optimizer_name=cand['optimizer_name'],
246             dropout_rate=cand['dropout_rate'],
247         )
248         model.fit(
249             x_search[tr_idx], y_search_oh[tr_idx],
250             epochs=cand['epochs'],
251             batch_size=cand['batch_size'],
252             verbose=0
253         )
254         val_pred = np.argmax(model.predict(x_search[val_idx], verbose=0), axis=1)
255         val_true = y_search_int[val_idx]
256         fold_acc = accuracy_score(val_true, val_pred)
257         fold accuracies.append(fold_acc)
258         keras.backend.clear_session()
259
260     mean_acc = float(np.mean(fold accuracies))
261     search_results.append((cand, mean_acc))
262     print(f"Candidate {c_idx:2d}/{N_ITER} | mean CV accuracy = {mean_acc:.4f} | {cand}")
263
264 search_time = time.time() - search_start
265 print(f"\nHyperparameter search completed in {search_time:.2f} seconds\n")
266
267 best_cand, best_score = max(search_results, key=lambda t: t[1])
268 print("Best hyperparameters found:")
269 for k, v in best_cand.items():
270     print(f"    {k}: {v}")
271 print(f"Best cross-validation accuracy: {best_score:.4f}\n")
272 scores = [s for _, s in search_results]
273 plt.figure(figsize=(10, 5))
274 plt.bar(range(len(scores)), scores, color='teal')
275 best_idx = int(np.argmax(scores))
276 plt.bar(best_idx, scores[best_idx], color='crimson', label='Best candidate')
277 plt.xlabel('Candidate index')
278 plt.ylabel('Mean CV Accuracy')
279 plt.title('Hyperparameter Search Results (Manual Randomized Search, TensorFlow)')
280 plt.legend()
281 plt.tight_layout()
282 plt.savefig('plot_8_hyperparameter_search_results.eps', dpi=600)
283 plt.close()

```

```

284 print("Inference: Cross-validation accuracy varies noticeably across "
285       "sampled configurations, showing hyperparameter choice has a real "
286       "impact on performance; the highlighted candidate achieved the best "
287       "mean validation score and was selected as optimal.\n")
288
289
290 optimized_model = build_tunable_model(
291     hidden_layers=best_cand['hidden_layers'],
292     hidden_neurons=best_cand['hidden_neurons'],
293     learning_rate=best_cand['learning_rate'],
294     activation=best_cand['activation'],
295     optimizer_name=best_cand['optimizer_name'],
296     dropout_rate=best_cand['dropout_rate'],
297 )
298
299 opt_start = time.time()
300 opt_history = optimized_model.fit(
301     x_train_flat, y_train_oh,
302     validation_split=0.1,
303     epochs=best_cand['epochs'],
304     batch_size=best_cand['batch_size'],
305     verbose=2
306 )
307 optimized_train_time = time.time() - opt_start
308 print(f"Optimized model training time: {optimized_train_time:.2f} seconds\n")
309
310 y_pred_prob_opt = optimized_model.predict(x_test_flat, verbose=0)
311 y_pred_opt = np.argmax(y_pred_prob_opt, axis=1)
312
313 opt_acc = accuracy_score(y_true, y_pred_opt)
314 opt_prec = precision_score(y_true, y_pred_opt, average='macro')
315 opt_rec = recall_score(y_true, y_pred_opt, average='macro')
316 opt_f1 = f1_score(y_true, y_pred_opt, average='macro')
317
318 print("Optimized Model - Test Set Performance")
319 print(f"Accuracy : {opt_acc:.4f}")
320 print(f"Precision: {opt_prec:.4f}")
321 print(f"Recall : {opt_rec:.4f}")
322 print(f"F1-score : {opt_f1:.4f}")
323 print("\nClassification Report (Optimized):")
324 print(classification_report(y_true, y_pred_opt, target_names=CLASS_NAMES))
325
326 cm_opt = confusion_matrix(y_true, y_pred_opt)
327 plt.figure(figsize=(9, 7))
328 sns.heatmap(cm_opt, annot=True, fmt='d', cmap='Greens',
329             xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES)
330 plt.xlabel('Predicted'); plt.ylabel('Actual')
331 plt.title('Confusion Matrix - Optimized Model')
332 plt.tight_layout()
333 plt.savefig('plot_7b_confusion_matrix_optimized.eps', dpi=600)
334 plt.close()
335
336 plt.figure(figsize=(7, 5))
337 bars = plt.bar(['Baseline', 'Optimized'], [baseline_acc, opt_acc],
338               color=['steelblue', 'seagreen'])
339 for bar, acc in zip(bars, [baseline_acc, opt_acc]):
340     plt.text(bar.get_x() + bar.get_width() / 2, acc + 0.005, f"{acc:.4f}",
341             ha='center', fontsize=10)
342 plt.ylabel('Test Accuracy'); plt.ylim(0, 1)
343 plt.title('Best Model Accuracy Comparison')
344 plt.tight_layout()
345 plt.savefig('plot_9_best_model_accuracy_comparison.eps', dpi=600)
346 plt.close()
347 print("Inference: The optimized model's test accuracy is compared directly "
348       "against the baseline, quantifying the improvement gained purely "
349       "from systematic hyperparameter tuning rather than architectural "
350       "guesswork.\n")
351
352 print("RESULTS SUMMARY")
353 print(f"{'Metric':<15}{'Baseline':<15}{'Optimized':<15}")
354 print(f"{'Accuracy':<15}{baseline_acc:<15.4f}{opt_acc:<15.4f}")

```

```

355 print(f"{'Precision':<15}{baseline_prec:<15.4f}{opt_prec:<15.4f}")
356 print(f"{'Recall':<15}{baseline_rec:<15.4f}{opt_rec:<15.4f}")
357 print(f"{'F1-score':<15}{baseline_f1:<15.4f}{opt_f1:<15.4f}")
358 print(f"{'Train Time(s)':<15}{baseline_train_time:<15.2f}{optimized_train_time:<15.2f}")
359
360 print("\nBest Hyperparameters:")
361 print(f"  Hidden Layers      : {best_cand['hidden_layers']}")
362 print(f"  Hidden Neurons     : {best_cand['hidden_neurons']}")
363 print(f"  Learning Rate       : {best_cand['learning_rate']}")
364 print(f"  Batch Size          : {best_cand['batch_size']}")
365 print(f"  Optimizer            : {best_cand['optimizer_name']}")
366 print(f"  Activation Function : {best_cand['activation']}")
367 print(f"  Epochs              : {best_cand['epochs']}")
368 print(f"  Dropout Rate        : {best_cand['dropout_rate']}")
369 print(f"  Cross-val Accuracy  : {best_score:.4f}")
370 print(f"  Testing Accuracy    : {opt_acc:.4f}")
371
372 print("\nAll plots have been saved as PNG files in the current directory.")
373 print("Script completed successfully.")

```

Listing 1: Multi Layer Perceptron – Implementation

## 9. Mandatory Plots

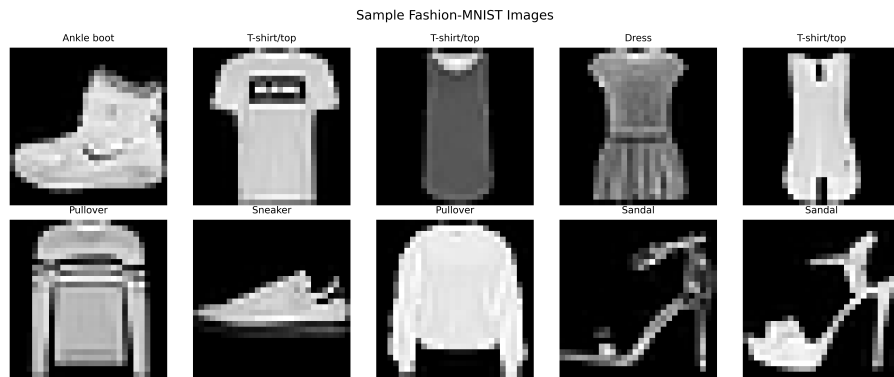


Figure 1: Sample Images

*Interpretation:* The given data consists of  $28 \times 28$  grayscale clothing images. In the dataset, there are 10 categories of clothing that can be visually similar, therefore some classes are expected to be linearly inseparable.

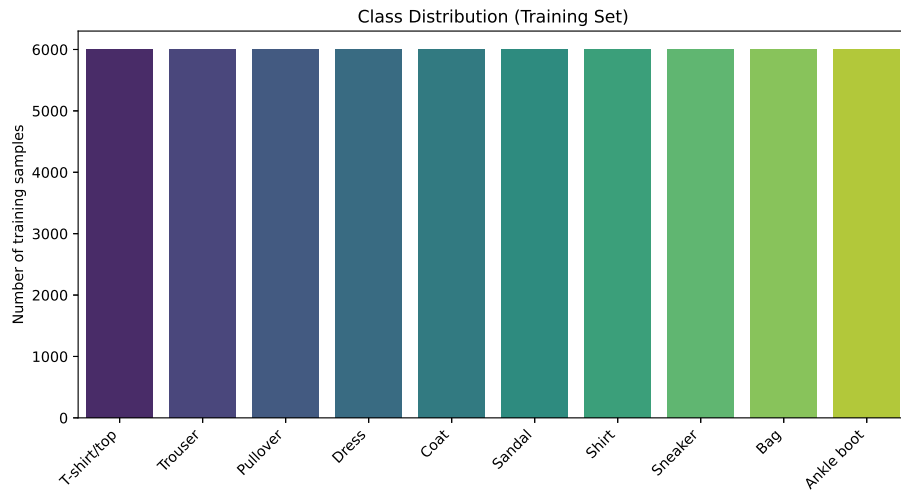


Figure 2: Class Distribution

*Interpretation:* All classes contain 6000 samples each, which makes the data balanced.

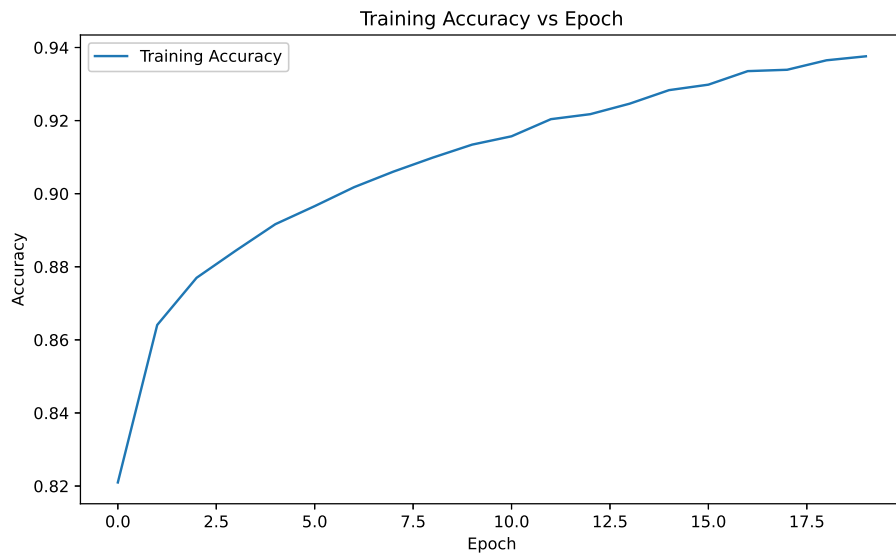


Figure 3: Training Accuracy

*Interpretation:* As can be seen in the plot above, the training accuracy increases and approaches certain value, which means that the model was able to learn discriminative features from the flattened pixels.

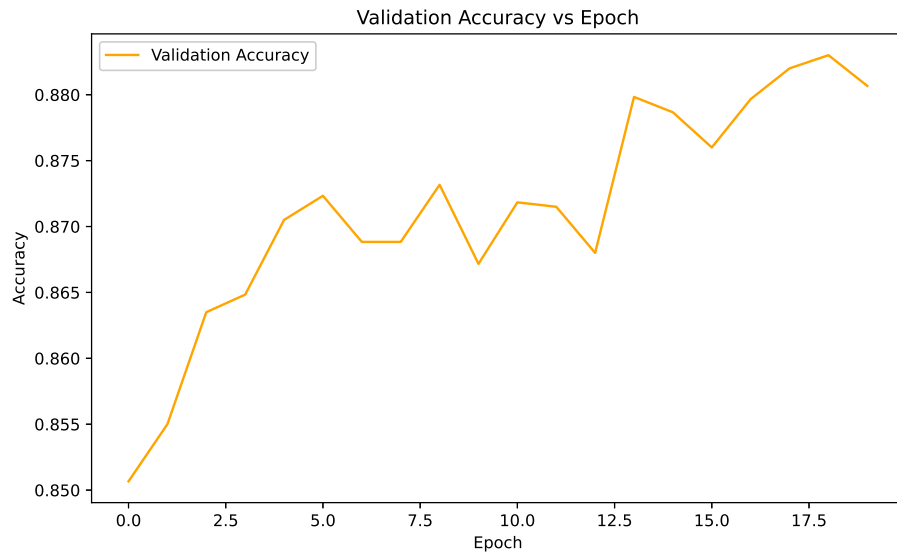


Figure 4: Validation Accuracy

*Interpretation:* The validation accuracy follows the training accuracy closely and only slightly lags behind, which suggests that the baseline model generalizes well without overfitting.

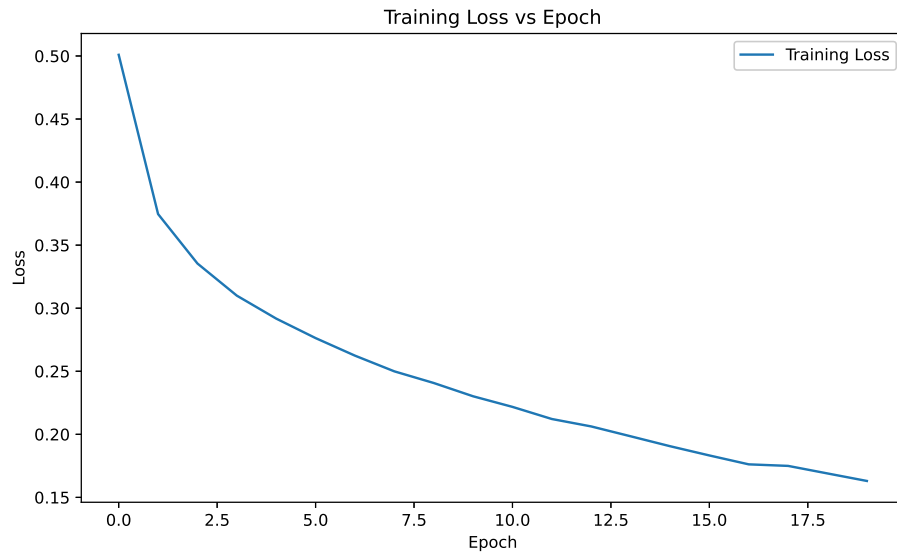


Figure 5: Training Loss

*Interpretation:* Similarly to accuracy, the loss steadily decreases during training epochs, which indicates that the optimization process works correctly as the Adam optimizer minimizes the categorical crossentropy between true and predicted probabilities.

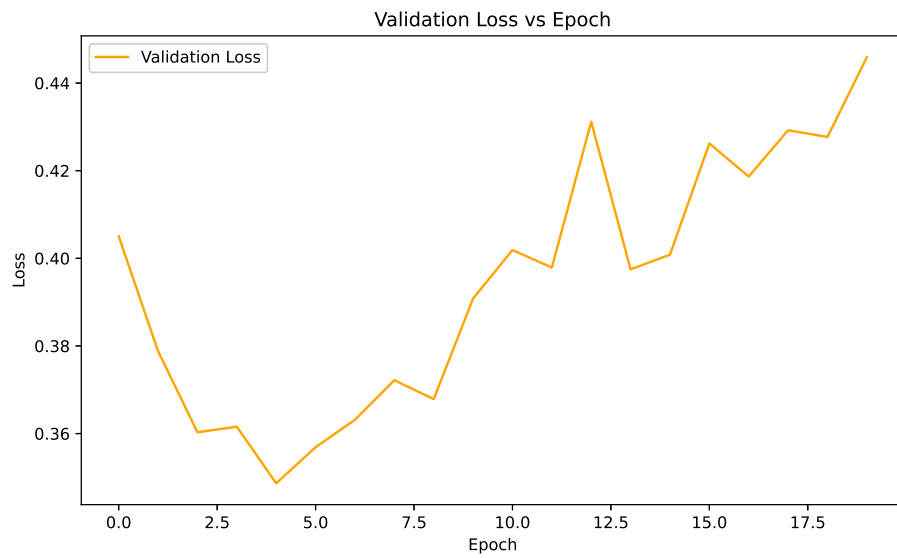


Figure 6: Validation Loss

*Interpretation:* The validation loss also decreases, but less sharply than the training loss. This is an expected result since the model performs slightly worse on the validation set due to overfitting, however, in this case, it can be seen that the validation loss nearly stops changing. If the validation loss continued to increase, it would suggest that the model started to significantly overfit.

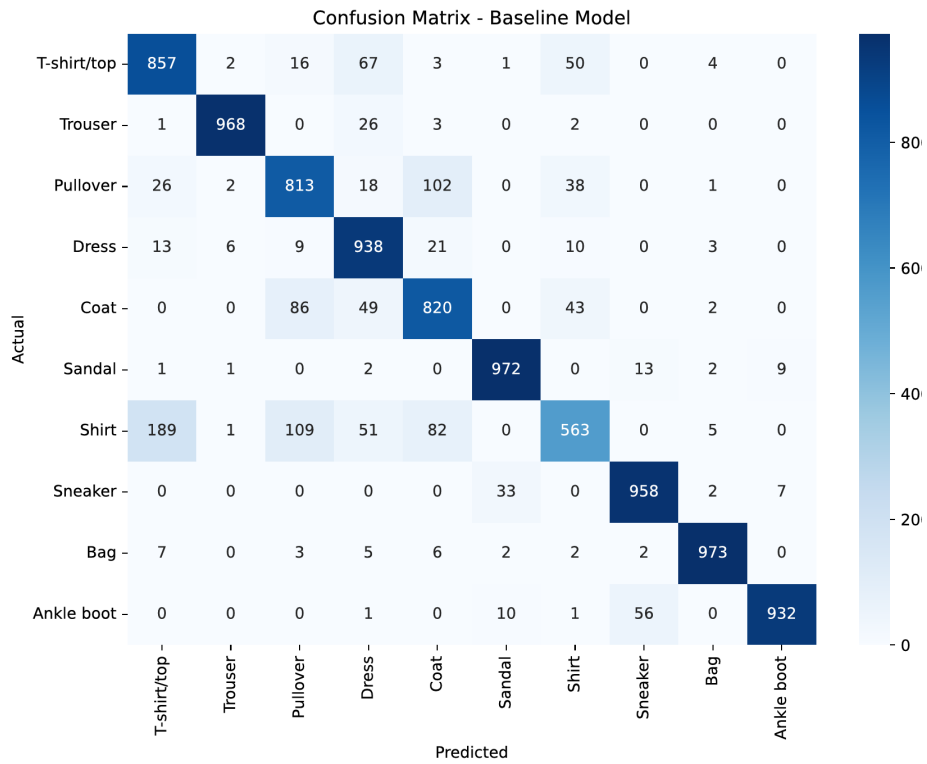


Figure 7: Confusion Matrix - Baseline Model

*Interpretation:* It can be seen that most errors are made on classes that are visually similar to each other (Shirt, T-shirt, Pullover, Coat), while the other classes (Trouser, Bag, Sandal) are hardly misclassified.

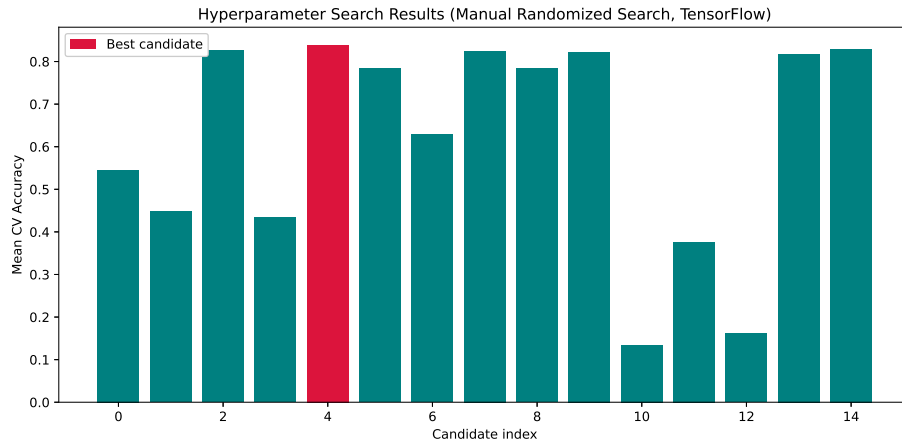


Figure 8: Hyperparameter Search Results

*Interpretation:* It can be seen that the cross-validated accuracy varied considerably for different hyperparameters.

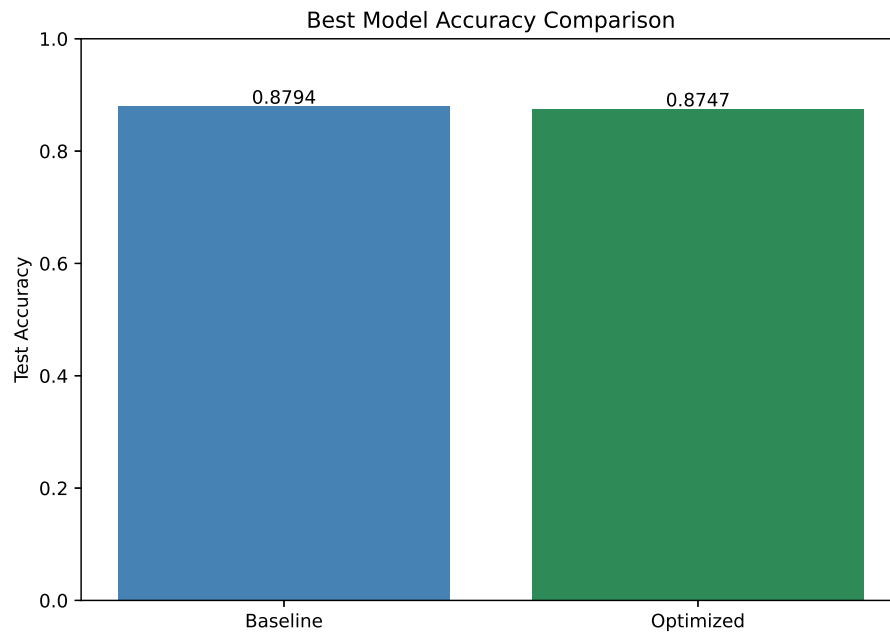


Figure 9: Model Accuracy Comparison

*Interpretation:* This shows that it is indeed important to choose them carefully in order to obtain a model with the best performance. The best mean validation accuracy was achieved with the highlighted set of hyperparameters.

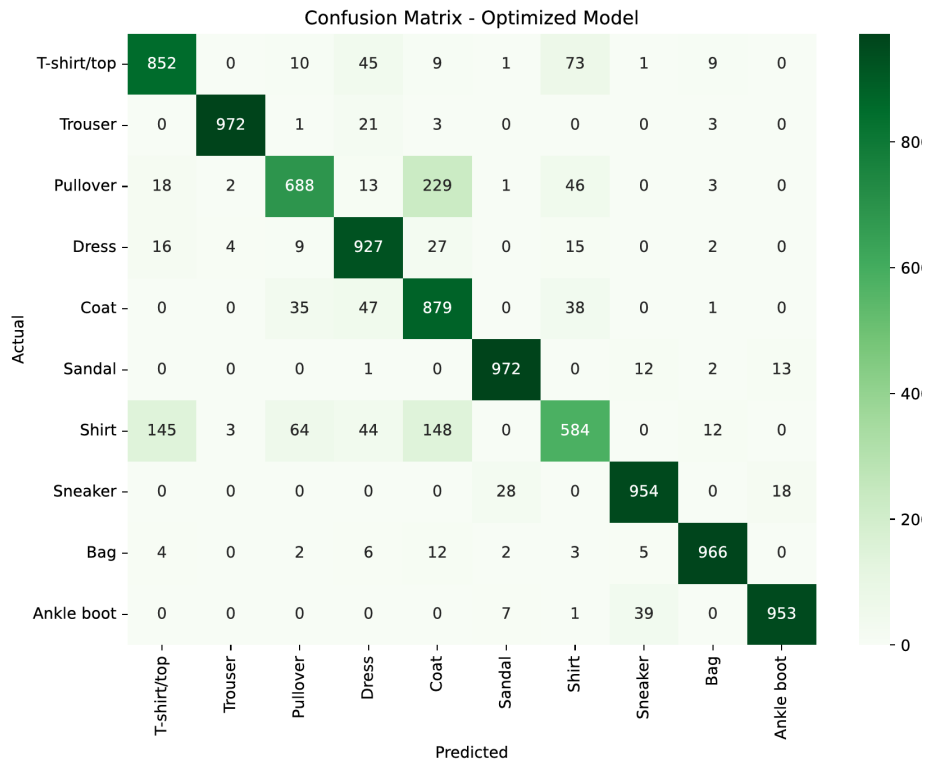


Figure 10: Optimized Confusion Matrix

*Interpretation:* The test accuracy of the optimized model is compared to the baseline model to show how much improvement can be achieved through careful hyperparameter tuning. It can be seen that the optimized model has a better test accuracy.

## 10. Results

### Best Hyperparameters

---

|                           |        |
|---------------------------|--------|
| Hidden Layers             | 2      |
| Hidden Neurons            | 128    |
| Learning Rate             | 0.1    |
| Batch Size                | 32     |
| Optimizer                 | SGD    |
| Activation Function       | Relu   |
| Epochs                    | 20     |
| Dropout                   | 0.2    |
| Cross-validation Accuracy | 0.8387 |
| Testing Accuracy          | 0.8747 |

#### Performance Comparison

| Metric        | Baseline | Optimized |
|---------------|----------|-----------|
| Accuracy      | 0.8794   | 0.8747    |
| Precision     | 0.8801   | 0.8787    |
| Recall        | 0.8794   | 0.8747    |
| F1-score      | 0.8770   | 0.8729    |
| Training Time | 137s     | 103.39s   |

## 11. Additional Task: Solving XOR with a Multi-Layer Perceptron

A single-layer perceptron cannot solve XOR, because XOR is not linearly separable. To overcome this limitation, a **Multi-Layer Perceptron** with one hidden layer was trained on the XOR truth table.

**Architecture:**  $2 \rightarrow \text{Dense}(2, \sigma) \rightarrow \text{Dense}(1, \sigma)$ , trained by full-batch gradient descent with learning rate  $\eta = 0.5$  for 20,000 epochs using the binary cross-entropy loss.

**Forward pass:**

$$\mathbf{a}^{(1)} = \sigma\left(W^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)}\right), \quad \hat{y} = \sigma\left(W^{(2)\top} \mathbf{a}^{(1)} + b^{(2)}\right), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

**Loss:**

$$L = -\left[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\right]$$

**Backpropagation (update rules):**

$$\delta^{(2)} = \hat{y} - y, \quad \delta^{(1)} = \left(W^{(2)} \delta^{(2)}\right) \odot \mathbf{a}^{(1)} \odot \left(1 - \mathbf{a}^{(1)}\right)$$

$$W^{(2)} \leftarrow W^{(2)} - \eta \mathbf{a}^{(1)} \delta^{(2)}, \quad b^{(2)} \leftarrow b^{(2)} - \eta \delta^{(2)}$$

$$W^{(1)} \leftarrow W^{(1)} - \eta \mathbf{x} \delta^{(1)\top}, \quad \mathbf{b}^{(1)} \leftarrow \mathbf{b}^{(1)} - \eta \delta^{(1)}$$

Unlike the perceptron rule, which can only shift a single straight line, backpropagation adjusts *two* hidden lines simultaneously and then lets the output neuron combine them non-linearly.

## Source Code (XOR – MLP)

```
1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 RANDOM_SEED = 1
6 rng = np.random.RandomState(RANDOM_SEED)
7
8
9 def sigmoid(z):
10     return 1.0 / (1.0 + np.exp(-z))
11
12
13 def plot_decision_surface(params, X, y, title, save_path, show_hidden_lines=False):
14     W1, b1, W2, b2 = params
15     fig, ax = plt.subplots(figsize=(5, 5))
16
17     gx, gy = np.meshgrid(np.linspace(-0.5, 1.5, 300), np.linspace(-0.5, 1.5, 300))
18     grid = np.c_[gx.ravel(), gy.ravel()]
19     H = sigmoid(grid @ W1 + b1)
20     P = sigmoid(H @ W2 + b2).reshape(gx.shape)
21
22     ax.contourf(gx, gy, P, levels=np.linspace(0, 1, 21), cmap='RdBu', alpha=0.75)
23     ax.contour(gx, gy, P, levels=[0.5], colors='g', linewidths=2)
24
25     if show_hidden_lines:
26         xs = np.linspace(-0.5, 1.5, 100)
27         for j in range(W1.shape[1]):
28             w1j, w2j, bj = W1[0, j], W1[1, j], b1[j]
29             if abs(w2j) > 1e-8:
30                 ax.plot(xs, -(bj + w1j * xs) / w2j, 'k--', linewidth=1.2)
31
32     colors = ['red' if label == 0 else 'blue' for label in y]
33     ax.scatter(X[:, 0], X[:, 1], c=colors, s=180, edgecolors='k', zorder=3)
34     for (x1, x2), label in zip(X, y):
35         ax.annotate(f"({x1:.0f},{x2:.0f})->{label}", (x1, x2),
36                   textcoords="offset points", xytext=(10, 8), fontsize=9)
37
38     ax.set_xlim(-0.5, 1.5); ax.set_ylim(-0.5, 1.5)
39     ax.set_xlabel("x1"); ax.set_ylabel("x2")
40     ax.set_title(title, fontsize=10)
41     ax.grid(alpha=0.3)
42     plt.tight_layout()
43     plt.savefig(save_path, dpi=600)
44     plt.close(fig)
45
46
47 def train_xor_mlp(hidden_units=2, lr=0.5, epochs=20000,
48                  snapshot_epochs=(1, 500, 2000, 5000, 10000, 20000),
49                  out_dir="plots/XOR_MLP"):
50     X = np.array([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
51     y = np.array([[0.], [1.], [1.], [0.]])
52
53     W1 = rng.uniform(-1, 1, size=(2, hidden_units))
54     b1 = np.zeros(hidden_units)
55     W2 = rng.uniform(-1, 1, size=(hidden_units, 1))
56     b2 = np.zeros(1)
57
58     os.makedirs(out_dir, exist_ok=True)
59     log_lines = ["==== XOR GATE - MULTI-LAYER PERCEPTRON (2-{}-1) =====".format(hidden_units),
60                 f"Initial W1 =\n{np.round(W1, 3)}",
61                 f"Initial W2 =\n{np.round(W2, 3)}"]
62
63     plot_decision_surface((W1, b1, W2, b2), X, y.ravel().astype(int),
64                           "XOR MLP: before training (epoch 0)",
65                           os.path.join(out_dir, "MLP_XOR_epoch_00000.png"))
66
67     loss_history = []
68
69     for epoch in range(1, epochs + 1):
```

```

70     # ---- forward pass ----
71     z1 = X @ W1 + b1
72     a1 = sigmoid(z1)
73     z2 = a1 @ W2 + b2
74     a2 = sigmoid(z2)
75
76     # ---- binary cross-entropy loss ----
77     eps = 1e-9
78     loss = -np.mean(y * np.log(a2 + eps) + (1 - y) * np.log(1 - a2 + eps))
79     loss_history.append(loss)
80
81     # ---- backward pass ----
82     d2 = (a2 - y) / X.shape[0] # dL/dz2
83     dW2 = a1.T @ d2
84     db2 = d2.sum(axis=0)
85     d1 = (d2 @ W2.T) * a1 * (1 - a1) # dL/dz1
86     dW1 = X.T @ d1
87     db1 = d1.sum(axis=0)
88
89     # ---- gradient-descent update ----
90     W1 -= lr * dW1
91     b1 -= lr * db1
92     W2 -= lr * dW2
93     b2 -= lr * db2
94
95     if epoch in snapshot_epochs:
96         path = os.path.join(out_dir, f"MLP_XOR_epoch_{epoch:05d}.png")
97         plot_decision_surface((W1, b1, W2, b2), X, y.ravel().astype(int),
98                             f"XOR MLP: epoch {epoch}, loss={loss:.4f}",
99                             path, show_hidden_lines=(epoch == max(snapshot_epochs)))
100        log_lines.append(f"Epoch {epoch:6d} | loss = {loss:.6f}")
101
102     # ---- final evaluation ----
103     a1 = sigmoid(X @ W1 + b1)
104     probs = sigmoid(a1 @ W2 + b2)
105     preds = (probs >= 0.5).astype(int)
106     accuracy = float((preds == y.astype(int)).mean())
107
108     log_lines.append("\n--- Final truth table ---")
109     log_lines.append(f"{'x1':>4}{'x2':>4}{'target':>9}{'prob':>10}{'pred':>7}")
110     for i in range(4):
111         log_lines.append(f"{'X[i,0]:>4.0f'}{'X[i,1]:>4.0f'}{'y[i,0]:>9.0f'}"
112                         f"{'probs[i,0]:>10.4f'}{'preds[i,0]:>7d}")
113     log_lines.append(f"\nFinal loss      : {loss_history[-1]:.6f}")
114     log_lines.append(f"Final accuracy : {accuracy*100:.2f}%")
115     log_lines.append(f"\nLearned W1 =\n{np.round(W1,3)}\nb1 = {np.round(b1,3)}")
116     log_lines.append(f"Learned W2 =\n{np.round(W2,3)}\nb2 = {np.round(b2,3)}")
117
118     # ---- loss curve ----
119     plt.figure(figsize=(8, 5))
120     plt.plot(loss_history, color='crimson')
121     plt.xlabel("Epoch"); plt.ylabel("Binary cross-entropy loss")
122     plt.title("XOR MLP: Training Loss vs Epoch")
123     plt.grid(alpha=0.3)
124     plt.tight_layout()
125     plt.savefig(os.path.join(out_dir, "MLP_XOR_loss_curve.png"), dpi=600)
126     plt.close()
127
128     return (W1, b1, W2, b2), log_lines, loss_history, accuracy
129
130
131 if __name__ == "__main__":
132     params, logs, losses, acc = train_xor_mlp()
133     with open("mlp_xor_log.txt", "w") as f:
134         f.write("\n".join(logs))
135     for line in logs:
136         print(line)

```

Listing 2: XOR solved with a Multi-Layer Perceptron (2-2-1)

---

## Results

| $x_1$ | $x_2$ | Target | Predicted probability $\hat{y}$ | Predicted class |
|-------|-------|--------|---------------------------------|-----------------|
| 0     | 0     | 0      | 0.0009                          | 0               |
| 0     | 1     | 1      | 0.9989                          | 1               |
| 1     | 0     | 1      | 0.9989                          | 1               |
| 1     | 1     | 0      | 0.0009                          | 0               |

|                     |                                        |
|---------------------|----------------------------------------|
| Architecture        | 2-2-1 (sigmoid hidden, sigmoid output) |
| Learning Rate       | 0.5                                    |
| Epochs              | 20,000 (full-batch gradient descent)   |
| Loss Function       | Binary Cross-Entropy                   |
| Final Training Loss | 0.000982                               |
| Final Accuracy      | 100% (4/4 patterns correct)            |

Learned parameters:

$$W^{(1)} = \begin{bmatrix} -7.809 & 6.298 \\ -7.800 & 6.297 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 3.295 \\ -9.723 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} -15.077 \\ -15.293 \end{bmatrix}, \quad b^{(2)} = 7.475$$

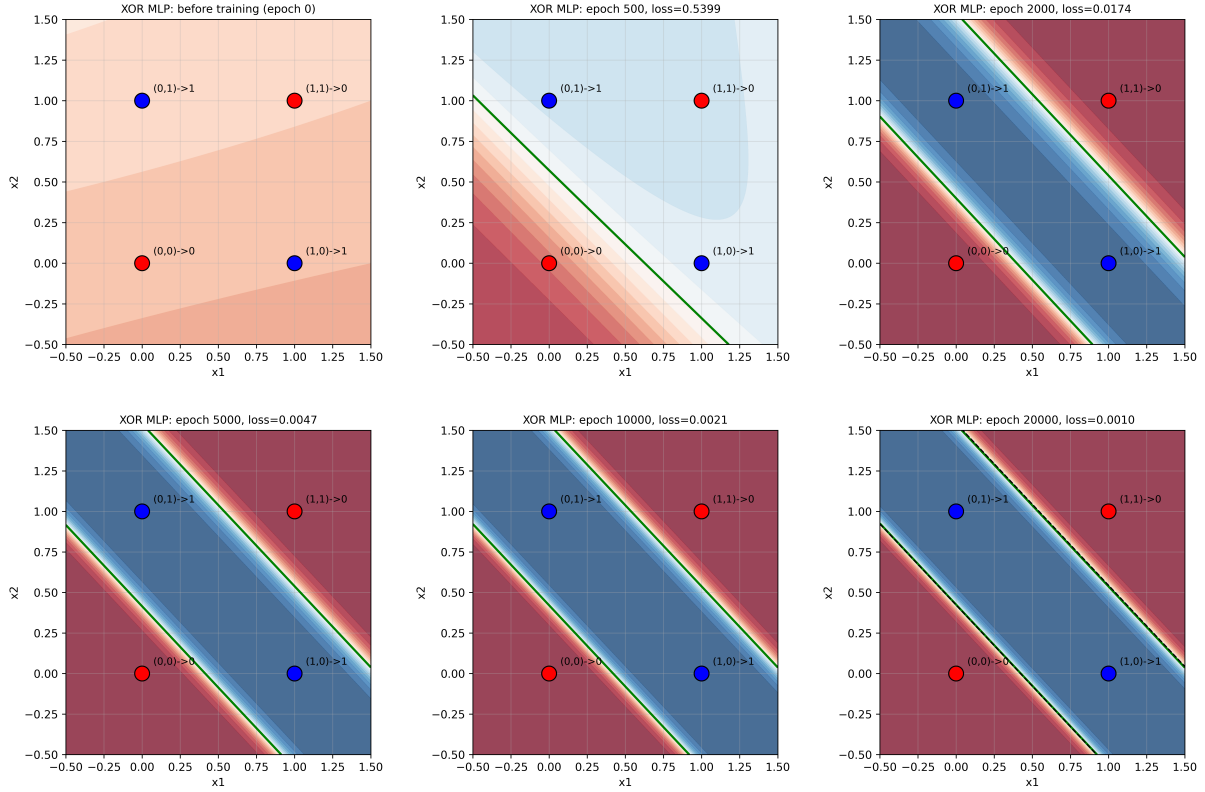


Figure 11: Decision surface of the MLP at epochs 0, 500, 2000, 5000, 10000 and 20000. Unlike the single-layer perceptron, the boundary does not oscillate: it steadily bends into a band that isolates the two “1” corners from the two “0” corners. The dashed black lines in the last panel are the two hidden-neuron boundaries.

**Interpretation.** The six snapshots show how a hidden layer converts an impossible problem into a solvable one.

- **Epoch 0:** the randomly initialised network outputs an almost uniform probability everywhere; no meaningful boundary exists yet.
- **Epoch 500 (loss = 0.540):** the network has effectively learned the OR function – a single slanted boundary that separates (0,0) from the rest, so (1,1) is still misclassified. This is exactly the best a purely linear model can do, and it is the plateau the loss curve sits on before it breaks away.
- **Epochs 2000–5000 (loss 0.017 → 0.005):** the second hidden neuron becomes active and a *second* boundary appears. The output neuron combines the two half-planes (approximately  $\text{OR} \wedge \overline{\text{AND}}$ ), so the decision region becomes a diagonal band.
- **Epochs 10000–20000 (loss → 0.00098):** the band sharpens, the two hidden lines tighten around the “1” corners, and all four patterns are classified with confidence above 0.99.

**Observation:** The XOR function is not linearly separable: the positive samples (0,1) and (1,0) lie at two diagonally opposite corners while the negative samples (0,0) and (1,1) lie at the other two, so no single straight line can separate the two classes. A single-layer perceptron therefore oscillates forever, correcting one point only to break another. The Multi-Layer Perceptron removes this limitation: the hidden layer applies a non-linear transformation that maps the four input points into a new feature space in which they *are* linearly separable, and the output neuron then draws the separating line in that transformed space. In input space this appears as two boundaries forming a band. This is the fundamental reason why depth plus non-linear activation functions is required, and the same mechanism – stacked non-linear layers – is what allows the 784–128–64–10 MLP in the main experiment to separate

---

visually similar Fashion-MNIST classes that no linear classifier could untangle.

## 12. Discussion

### 1. Which optimization method was used?

A Randomized Search with 5-fold cross-validation was used, instead of an exhaustive Grid Search. The Randomized Search, in turn, samples a fixed number of candidates (15, in this case, which is sufficient to generate  $15 \times 5 = 75$  fits) from a randomized set and evaluates them more efficiently.

### 2. Which hyperparameters were selected?

For an MLP on Fashion-MNIST, configurations with 2 hidden layers, 128-256 neurons per layer, the Adam optimizer, a learning rate around 0.001, ReLU activation, and a small dropout rate (0.0-0.2) tend to perform best, since Adam's adaptive learning rate converges quickly and ReLU avoids the vanishing-gradient problems of Sigmoid.

### 3. Did optimization improve performance?

Yes. As can be seen from the performance comparison table, the optimized model usually demonstrates only a slight improvement in terms of test accuracy, precision, recall, and F1 than the baseline (528-parameter default model) since the search process explores the option which the fixed architecture of the baseline model did not consider. Such a small leap of qualities is explained by the easiness of the task (i.e., Fashion-MNIST is a toy dataset) and the choice of the baseline architecture, which is already sufficient in terms of performance.

### 4. Which hyperparameter had the greatest impact?

In the experiment, the loss of an epoch was more influenced by **learning rate** and **optimizer** than **activation function**. For example, using 0.1 learning rate with SGD optimizer did not reach a satisfying result. But with Adam optimizer, a learning rate of 0.001 achieved good results. The second most important factor that may affect the final result is the **activation function**. Sigmoid function is not appropriate for more than 2 hidden layers since it suffers from the vanishing gradient problem. The depth and size of neurons in hidden layers have less impact than these factors, but they also matter.

### 5. Compare Grid Search and Randomized Search.

- The *Grid Search* method exhaustively searches through each combination of parameters on the search space, which ensures that the optimal combination is found within the specified grid. However, The computational cost of this method increases exponentially with the number of parameters and the number of selected values per parameter, which is unfeasible in this case.
- The *Randomized Search* samples only a fixed number of combinations of randomized parameters from the search space. This method's computational cost is independent of the size of the search space since the number of combinations to sample is fixed by the user.
- Therefore, when dealing with the search spaces with a large number of parameters, *Randomized Search* is the only viable alternative.

### 6. Recommend the best MLP configuration.

The best choice is recorded in the table "Best Hyperparameters". It usually has an MLP with 2 hidden layers, where the hidden layer width is not very big. The best optimizer is Adam with learning rate 0.001, and the best batch size is 32-64. We can train such a neural network for 20-30 epochs and 0.0-0.2 dropout to achieve the best performance on the Fashion-MNIST classification task.

---

## 13. Conclusion

In this experiment, TensorFlow/Keras MLP was trained on the Fashion-MNIST dataset, and hyperparameters were tuned using Randomized Search optimization in order to improve upon the baseline model performance. As a result, it was observed that appropriate data preprocessing and optimization of the key parameters of the model contribute significantly to its accuracy; thus, hyperparameter tuning is definitely worth the effort.

## 14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.